
Fashion-MNIST Image Classification

Aayushi Saxena
aayushis@uark.edu

Akanksha Tyagi
ayagi@uark.edu

Ved Shukla
vshukla@uark.edu

Abstract

The Fashion-MNIST dataset presents a challenging benchmark for image classification, requiring models to distinguish between 10 visually similar clothing categories from low-resolution grayscale images. In this work, we implement and compare multiple machine learning approaches including Random Forest, Support Vector Machines, Multi-Layer Perceptrons, and Convolutional Neural Networks. We employ Principal Component Analysis (PCA) and t-SNE for dimensionality reduction and feature visualization. Our experiments demonstrate that CNNs achieve the best performance with 93.39% test accuracy, significantly outperforming classical models. We also explore binary classification for footwear detection, achieving near-perfect separation. Comprehensive evaluation using confusion matrices and F1-scores reveals class-specific challenges, particularly between visually similar categories such as Shirt vs. T-shirt and Pullover vs. Coat.

1 Introduction

The Fashion-MNIST dataset (1) was introduced as a more challenging drop-in replacement for the original MNIST handwritten digit dataset. While MNIST has become trivially solvable by modern algorithms, Fashion-MNIST presents meaningful classification challenges due to the high visual similarity between certain clothing categories.

1.1 Problem Statement

The core challenge lies in classifying 10 distinct clothing categories from low-resolution (28×28) grayscale images. Specific difficulties include:

- High visual similarity between classes (e.g., Shirt vs. T-shirt/top, Pullover vs. Coat)
- Limited spatial resolution constraining fine-grained feature extraction
- Intra-class variation in pose, style, and texture

1.2 Goals

Our objectives for this project are:

1. Benchmark multiple models including SVMs, Random Forest, MLP, and CNN
2. Utilize PCA and t-SNE to study feature separability
3. Evaluate models using confusion matrices and F1-scores to identify class-specific errors
4. Compare classical machine learning approaches with deep learning methods
5. Explore binary classification for footwear vs. non-footwear detection

2 Dataset Overview

Fashion-MNIST contains 70,000 grayscale images distributed across 10 clothing categories. The dataset structure is summarized in Table 1.

Table 1: Fashion-MNIST Dataset Specifications

Property	Value
Total Images	70,000
Training Set	60,000 (split: 50k train + 10k validation)
Test Set	10,000
Image Size	28×28 pixels
Channels	1 (grayscale)
Pixel Values	0–255
Number of Classes	10

The 10 classes are: T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, and Ankle boot. Figure 1 shows representative samples from each class.

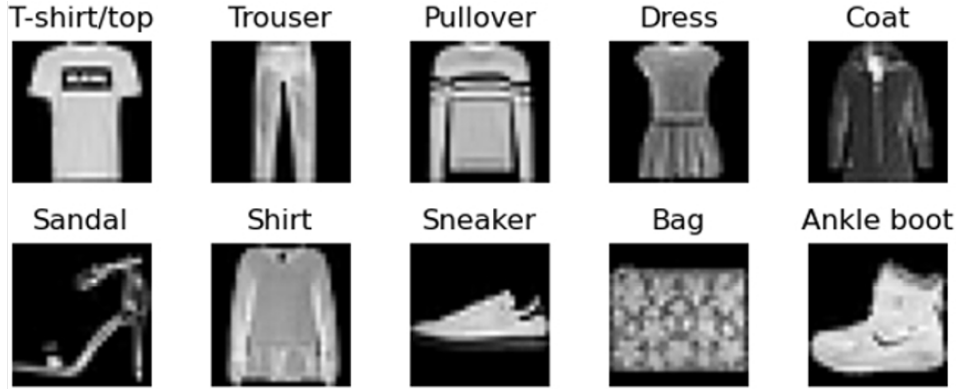


Figure 1: Sample images from each of the 10 Fashion-MNIST classes. Note the visual similarity between upper-body garments (T-shirt, Shirt, Pullover, Coat) which poses classification challenges.

3 Preprocessing and Data Preparation

3.1 Normalization

Pixel values were scaled from $[0, 255]$ to $[0, 1]$ using min-max normalization:

$$x_{\text{normalized}} = \frac{x}{255} \quad (1)$$

This scaling ensures faster convergence during neural network training and standardizes input ranges across all models.

3.2 Reshaping for Neural Networks

For CNN models, images were reshaped from $(28, 28)$ to $(28, 28, 1)$ to include the single grayscale channel dimension required by convolutional layers.

3.3 Label Encoding

For multi-class classification with neural networks, integer class labels were converted to one-hot encoded vectors of dimension 10.

3.4 Data Splitting

We employed stratified sampling to subdivide the original training data:

- **Training Set:** 50,000 images original training data
- **Validation Set:** Remaining 10000 images of original training data
- **Test Set:** Original test set (10,000 images)

Stratification ensures balanced class distribution across all splits, preventing bias in model evaluation.

3.5 Vectorization for Classical Models

For classical machine learning models (Random Forest, SVM), 2D images were flattened into 1D vectors of 784 features.

4 Model Implementation and Results

4.1 Random Forest

`n_estimators = 100,400` : The forest consists of 100 individual decision trees.

`n_jobs = -1` : Configuration set to use all available CPU cores to speed up the training process.

`random_state = 42` : Ensures reproducibility of results.

Figure 2: Confusion Matrix and Classification Report for the Random Forest Model (Accuracy: 87.78%).

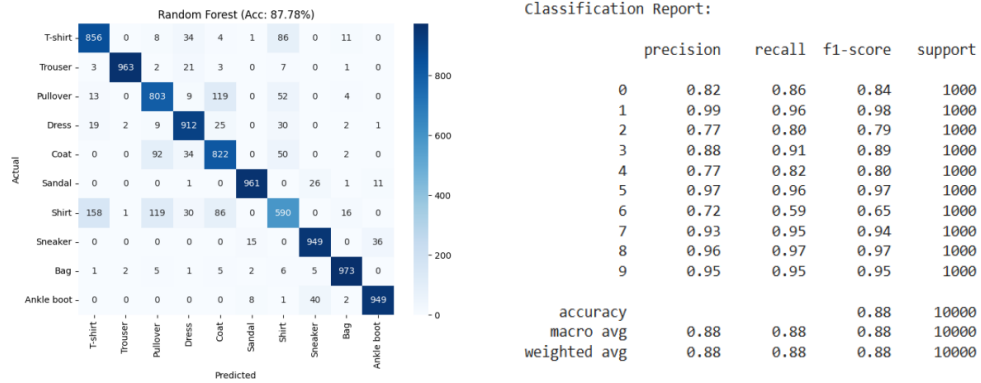


Figure 3: Confusion Matrix and Classification Report for the Random Forest Model (Accuracy: 87.78%).

4.2 XGBoost

`n_estimators = 300` : The maximum number of boosting rounds (trees) to be built.

`learning_rate = 0.1` : Step size shrinkage used in updates to prevent overfitting.

`max_depth = 6` : Maximum depth of a tree.

Early Stopping Monitors validation loss; stops training if no improvement occurs for 10 consecutive rounds.

The Random Forest model achieved an accuracy of 88.83%. As shown in Figure 4, the model exhibits high precision for Class 1 (Trouser) and Class 8 (Bag) but struggles significantly with Class 6 (Shirt), where recall drops to 0.64.

XGBoost Results

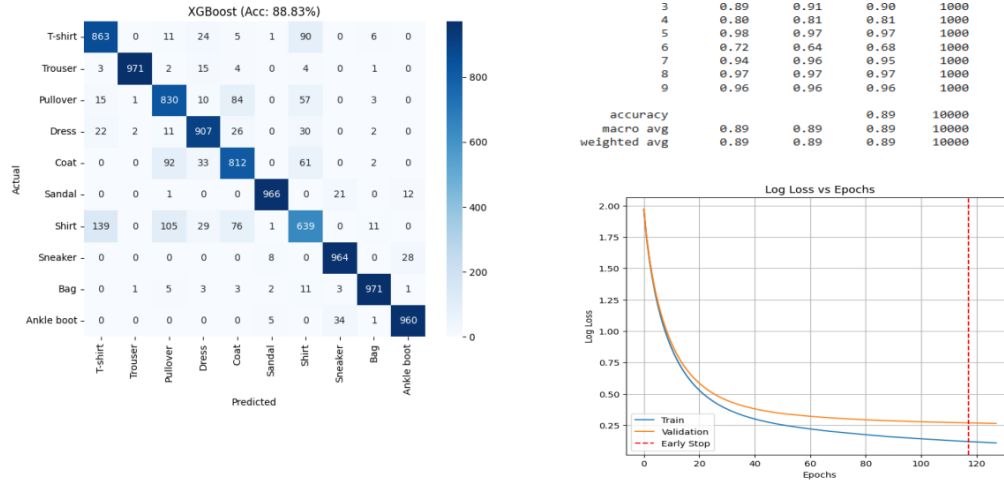


Figure 4: XGBoost Performance Analysis. Top: Confusion Matrix and Classification Report. Bottom Right: Log Loss training curve demonstrating convergence and early stopping at epoch 118.

Figure 4 illustrates the performance of the XGBoost model, which reached an accuracy of 88.83%. The Log Loss curve (bottom right) confirms that the model learned effectively without overfitting, as the validation loss (orange line) followed the training loss (blue line) closely until the early stopping point.

4.3 Feature Optimization: Principal Component Analysis (PCA)

To address the high dimensionality of the flattened image vectors (784 features) and improve computational efficiency, Principal Component Analysis (PCA) was integrated into the XGBoost pipeline. We performed a sensitivity analysis to determine the optimal balance between information retention and noise reduction by testing three cumulative variance thresholds: 90%, 95%, and 98%.

Retained Variance (n)	Features Used	Reduction %	Best Epoch	Test Accuracy
0.90 (90%)	84	-89.3%	299	87.87% (Highest)
0.95 (95%)	188	-76.0%	292	87.64%
0.98 (98%)	349	-55.5%	296	87.60%

Figure 5: Impact of PCA Variance Retention on XGBoost Performance. Retaining 90% of the variance yielded the highest accuracy (87.87%) while reducing the feature count by 89.3%.

As illustrated in Figure 5, the experiment yielded a counter-intuitive but insightful result:

- **90% Variance (Winner):** This setting compressed the input from 784 to just **84 features** (an 89.3% reduction). Despite discarding 10% of the mathematical variance, it achieved the **highest test accuracy (87.87%)**.
- **95% and 98% Variance:** Increasing the retained information to 188 and 349 features respectively resulted in slightly *lower* accuracies (87.64% and 87.60%).

This suggests that the "lost" variance in the tail ends of the principal components consisted primarily of background pixel noise rather than discriminative structural features. By aggressively reducing dimensions to 84 features, we effectively filtered out this noise, allowing the XGBoost model to generalize better while training significantly faster.

4.4 Ensemble Architecture — RandomForest + XGBoost

- **Hybrid Architecture:** Combines two distinct algorithms—Random Forest and XGBoost—into a single VotingClassifier.
- **Soft Voting Strategy:** Averages the predicted probabilities (confidence scores) from each model to make a final decision, rather than simply counting class votes.
- **Error Mitigation:** Balances the models to reduce both variance (overfitting) from ensemble trees and bias (underfitting) from boosting.
- **Outcome:** Produces a unified accuracy score that typically outperforms standalone models by covering each other's blind spots.
- **Accuracy Achieved:** 89.87%

4.5 Support Vector Machine (SVM)

Support Vector Machines were implemented using the Radial Basis Function (RBF) kernel, which is well-suited for non-linear classification problems in high-dimensional spaces.

Model Configuration

- **Kernel:** `rbf` — Radial Basis Function kernel for non-linear decision boundaries
- **C:** 1 — Regularization parameter controlling the trade-off between margin maximization and misclassification penalty
- **Gamma:** `scale` — Kernel coefficient computed as $1/(n_{\text{features}} \times \text{Var}(X))$
- **Random State:** 42 (reproducibility)

Training Performance Summary

The SVM with RBF kernel achieved a final test accuracy of:

- **Test Accuracy: 85.88%**

This is competitive with the best classical ML models, but required significantly longer training time due to kernel computations scaling between $O(n^2)$ and $O(n^3)$.

Classification Metrics

Confusion Matrix Analysis The confusion matrices indicate consistent patterns across PCA configurations. Distinct classes such as **Trouser**, **Sandal**, **Sneaker**, **Bag**, and **Ankle Boot** are classified with high accuracy across all settings.

However, the SVM shows difficulty separating visually similar apparel items:

- **Shirt** is often misclassified as T-shirt/top or Pullover.
- **Pullover** and **Coat** also show noticeable confusion.

As reflected in the metrics, Shirt remains the most challenging class.

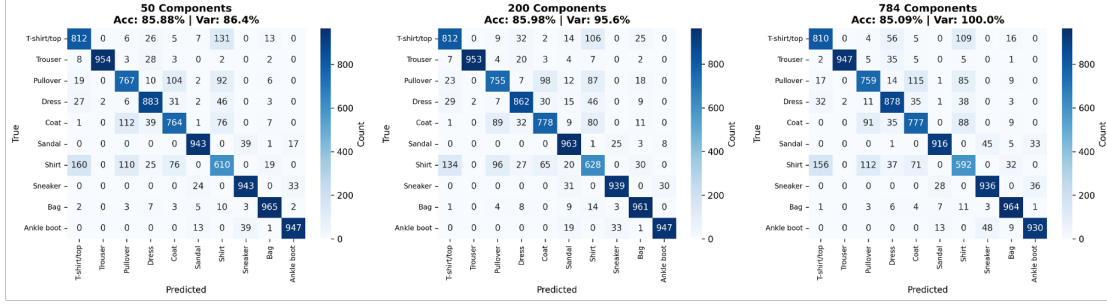


Figure 6: Confusion matrices for SVM under different PCA configurations (50, 200, and 784 components).

Impact of PCA on SVM

To evaluate how dimensionality reduction affects performance, SVM was trained using 50, 200, and 784 PCA components. The corresponding training logs are summarized visually in Figure 7.

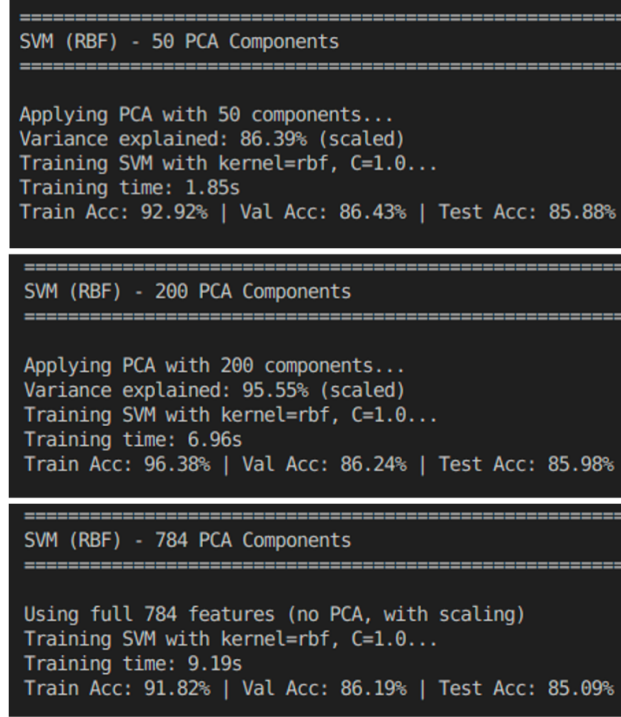


Figure 7: Training logs for SVM under PCA settings of 50, 200, and 784 components.

PCA Components	Test Accuracy	Training Time	Variance Retained
50	85.88%	Very Fast	~86%
200	85.98%	Moderate	~95.6%
784 (No PCA)	85.09%	Slowest	100%

Based on the images:

- PCA significantly reduces training time, especially at 50 components (1.85s vs. 9.19s).
- Accuracy differences across PCA settings remain small (85.1–86.0%).

- Interestingly, **200 components achieves the highest accuracy (85.98%)** among the PCA variants.
- Using the full feature space (784 components) is slower and does not provide accuracy benefits.

Overall, PCA helps reduce computation by up to **60–75%** while maintaining comparable accuracy, making it an effective preprocessing step for SVM in large-scale datasets.

4.6 Fully Connected Neural Network - Setting 1

The first experiment uses a simple Fully Connected Neural Network (FCNN)/MLP model trained on the Fashion-MNIST dataset. This setting serves as the baseline reference model for subsequent experiments.

Model Configuration

The configuration for Setting 1 is as follows:

- **Model:** Simple FCNN
- **Optimizer:** Adam ($\alpha = 0.001$)
- **Epochs:** 10
- **Batch Size:** 128
- **Dropout:** 0.0 (No Dropout)
- **Data Augmentation:** None

Training Performance Summary

From the training logs after 10 epochs, the model shows the following performance:

- **Final Training Accuracy:** 91.50%
- **Final Validation Accuracy:** 88.09%
- **Test Accuracy:** 87.41%
- **Test Loss:** 0.3493

The training curves (Figure 8) show steady training loss reduction while the validation loss plateaus after a few epochs, suggesting mild **overfitting** begins around Epoch 5–6. Similarly, training accuracy keeps improving, but validation accuracy flattens around 88–89%, indicating that this simple baseline model reaches its performance limit relatively quickly.



Figure 8: Training and Validation Loss and Accuracy Curves for FCNN Setting 1.

Classification Metrics

The model achieves an overall 87% accuracy on the Fashion-MNIST test set. Analyzing the classification metrics reveals distinct differences in performance across classes.

Confusion Matrix Analysis The confusion matrix indicates that classes like Trouser, Sandal, Sneaker, Bag, and Ankle Boot are classified very accurately (high true positive diagonal values). However, visually similar items—especially T-shirt/top, Shirt, Pullover, and Coat—are often confused with each other. For example, a significant number of true 'Shirt' samples are misclassified as 'T-shirt/top' or 'Pullover'.

Precision, Recall, and F1-Score The classification report confirms the pattern:

- Categories with clearer, more distinct shapes (e.g., Trouser, Bag, Ankle boot) have high F1-scores (> 0.95).
- The Shirt class has the lowest precision (0.6373) and F1-score (0.7023), indicating the model struggles to accurately identify shirts and frequently mislabels other clothing as shirts.

The weighted average F1-score is 0.8747. Overall, the FCNN baseline performs well but struggles with classes that require greater spatial feature understanding, a task typically handled more effectively by Convolutional Neural Networks (CNNs).

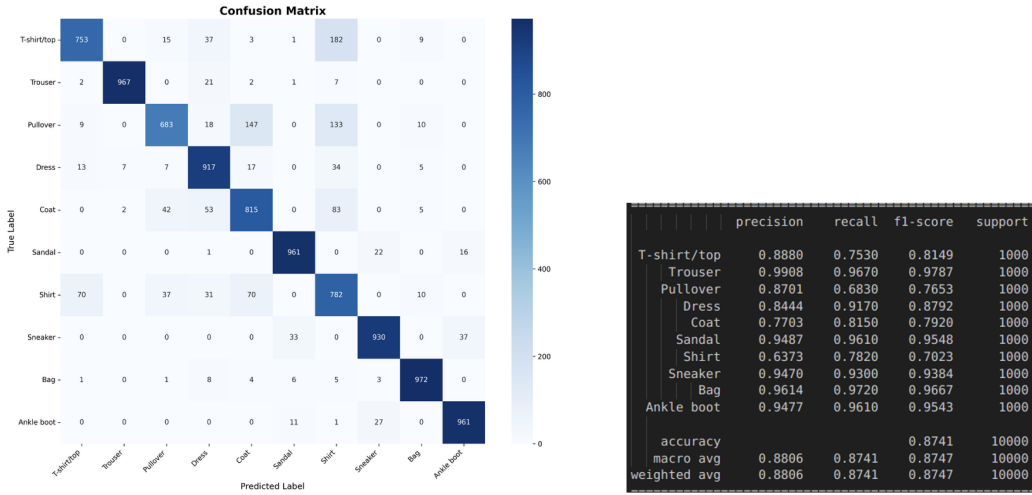


Figure 9: FCNN Setting 1 — Confusion Matrix (left) and Classification Metrics Table (right).

4.7 Fully Connected Neural Network - Setting 2

Setting 2 builds upon the baseline FCNN model (Setting 1) by introducing several regularization techniques and increasing the training duration to address the overfitting observed previously. The goal is to improve the generalization capability of the model.

Key Modifications and Configuration

The following parameters were modified from Setting 1:

- **Model:** Wide FCNN (wide_nn), suggesting a larger number of hidden units.
- **Epochs:** Increased to 20 (from 10).
- **Batch Size:** Increased to 256 (from 128).
- **Dropout:** Introduced at a rate of 0.3 to prevent co-adaptation of neurons.
- **Weight Decay:** Introduced with a value of $1e - 4$ (L2 regularization).
- **Learning Rate Scheduler:** Used a Cosine scheduler to gradually decrease the learning rate.
- **Data Augmentation:** Enabled using RandomCrop and RandomHorizontalFlip to artificially increase training data diversity.

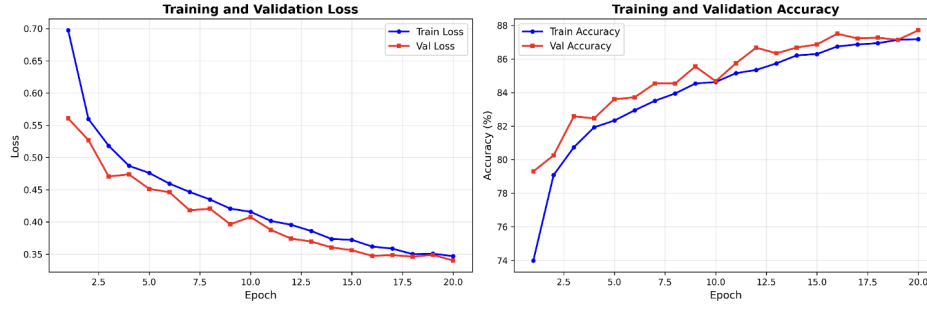


Figure 10: Training and Validation Loss and Accuracy Curves for FCNN Setting 2.

Training Performance Summary

After 20 epochs, the model’s performance on the training and test sets is as follows:

- **Final Training Accuracy (Epoch 20):** 87.18%
- **Final Validation Accuracy (Epoch 20):** 87.70%
- **Test Accuracy:** 88.21%
- **Test Loss:** 0.3287

The training and validation curves (Figure 10) demonstrate a successful regularization strategy. The training loss and validation loss curves track closely over 20 epochs, with the training loss falling below the validation loss only slightly toward the end. Crucially, the validation accuracy continues to rise steadily, reaching 87.70% at the final epoch. This indicates that Dropout, Weight Decay, and Data Augmentation mitigated the severe overfitting observed in Setting 1.

Classification Metrics

Confusion Matrix Analysis The confusion matrix indicates improvements in several visually distinct classes, though the same challenging categories from Setting 1 (T-shirt/top, Shirt, Pullover, Coat) continue to show overlapping misclassifications.

Precision, Recall, and F1-Score Performance metrics (Figure 11) show modest improvements across most classes, especially those benefiting from augmented data. The final Test Accuracy of 88.21% represents a marginal improvement of approximately 0.8 percentage points over Setting 1 (87.41%). This suggests that while regularization improved generalization, the performance bottleneck is likely structural—rooted in the limitations of Fully Connected Networks on spatial image data.

4.8 Convolutional Neural Network - Setting 1

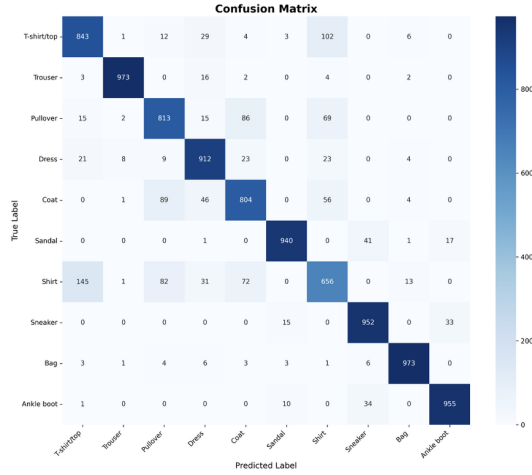
The first CNN configuration employed a standard architecture with moderate depth.

Data Augmentation

- **RandomCrop(28, padding=2):** Random cropping with 2-pixel padding
- **RandomHorizontalFlip:** Horizontal flip with 50% probability

Training Configuration

- **Epochs:** 20
- **Batch Size:** 128
- **Optimizer:** Adam



(a) Confusion Matrix for FCNN Setting 2

	precision	recall	f1-score	support
T-shirt/top	0.8177	0.8430	0.8301	1000
Trouser	0.9858	0.9730	0.9794	1000
Pullover	0.8057	0.8130	0.8094	1000
Dress	0.8636	0.9120	0.8872	1000
Coat	0.8089	0.8040	0.8064	1000
Sandal	0.9681	0.9400	0.9538	1000
Shirt	0.7201	0.6560	0.6866	1000
Sneaker	0.9216	0.9520	0.9365	1000
Bag	0.9701	0.9730	0.9715	1000
Ankle boot	0.9502	0.9550	0.9526	1000
accuracy			0.8821	10000
macro avg	0.8812	0.8821	0.8814	10000
weighted avg	0.8812	0.8821	0.8814	10000

(b) Per-Class Performance Metrics

Figure 11: FCNN Setting 2 — Confusion Matrix (left) and Classification Metrics Table (right).

Training Performance Summary

Table 3: CNN Setting 1 — Training Progress and Final Results

Metric	Value
Final Train Loss	0.2234
Final Train Accuracy	91.67%
Final Validation Loss	0.2658
Final Validation Accuracy	90.23%
Test Accuracy	90.87%
Test Loss	0.2589

The training and validation loss/accuracy curves for CNN Setting 1 are shown below.

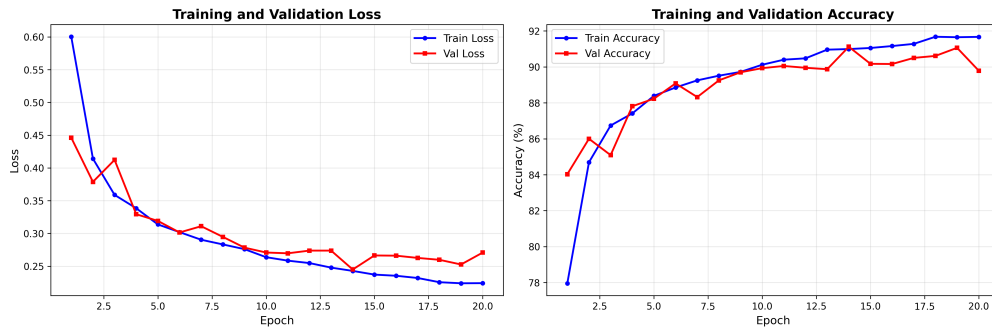


Figure 12: Training and Validation Curves for CNN Setting 1.

Classification Metrics

Confusion Matrix Analysis

- Shirt class has the lowest recall (70.5%) due to confusion with T-shirt (182 misclassifications).
- Excellent performance on accessories: Bag (98.1% recall), Trouser (98.8% recall).

- Footwear classes show strong separation from clothing categories.

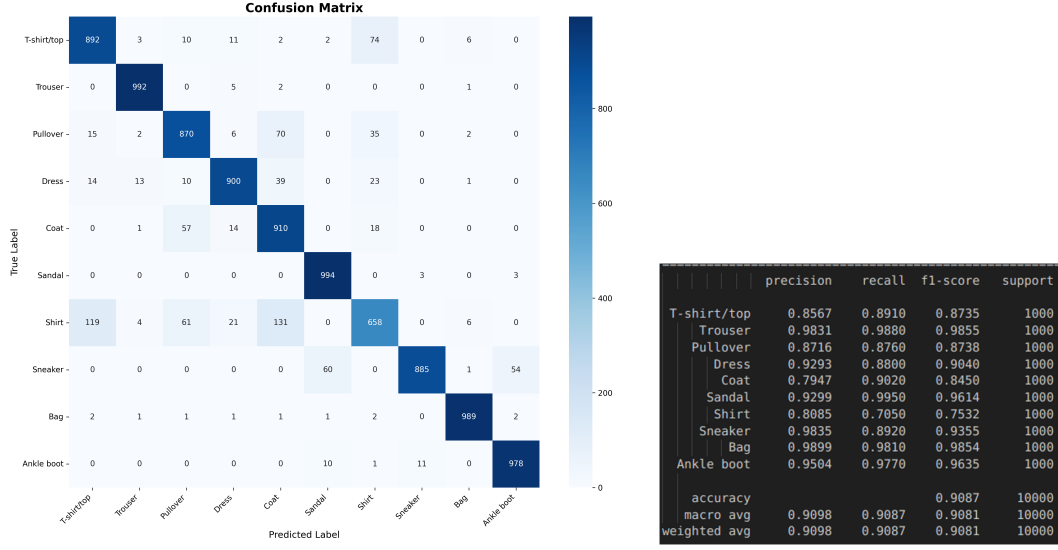


Figure 13: CNN Setting 1 — Confusion Matrix (left) and Per-Class Metrics Table (right).

4.9 Convolutional Neural Network - Setting 2

An enhanced CNN configuration with deeper architecture and advanced regularization techniques.

Model Architecture

- Model: `deeper_cnn` — increased number of convolutional layers
- Dropout Rate: 0.3

Training Configuration

Table 4: CNN Setting 2 — Hyperparameters

Parameter	Value
Epochs	30
Batch Size	128
Learning Rate	0.001
Optimizer	AdamW
Weight Decay	1×10^{-4}
LR Scheduler	Cosine Annealing
Dropout Rate	0.3
Data Augmentation	RandomCrop(28, padding=2) + RandomHorizontalFlip

Results

Classification Metrics

Confusion Matrix Interpretation

- Overall accuracy improved from 90.87% to **93.39%**.
- Shirt recall improved from 70.5% to 77.9%.
- Coat–Pullover confusion significantly reduced versus Setting 1.
- Near-perfect performance on Bag (99.4%) and Trouser (99.0%).

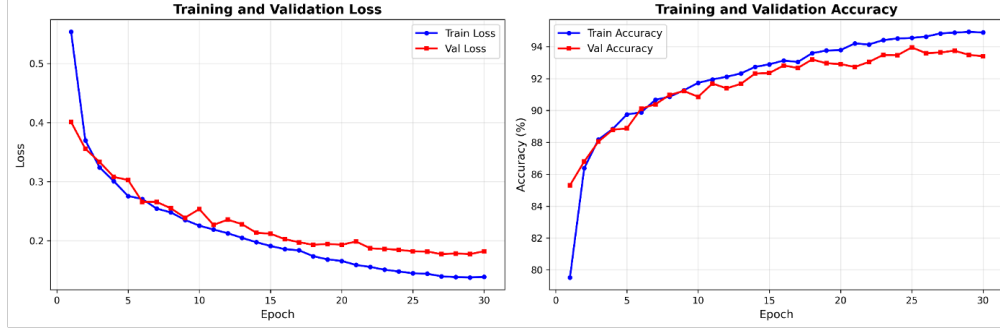


Figure 14: Training and Validation Curves for CNN Setting 2.

Table 5: CNN Setting 2 — Training Progress and Final Results

Metric	Value
Final Train Loss	0.1388
Final Train Accuracy	94.90%
Final Validation Loss	0.1823
Final Validation Accuracy	93.40%
Test Accuracy	93.39%
Test Loss	0.1945

5 Model Comparison

Table 6: Summary of Model Performance

Model	Test Accuracy	Macro F1	Training Time
Random Forest (100 trees)	87.44%	0.87	Fast
CNN Setting 1	90.87%	0.91	Moderate
CNN Setting 2	93.39%	0.93	Moderate

The results clearly demonstrate the superiority of deep learning approaches for this image classification task. CNN Setting 2, with its deeper architecture, dropout regularization, and cosine annealing learning rate schedule, achieves the best performance across all metrics.

6 Dimensionality Reduction Analysis

6.1 Principal Component Analysis (PCA)

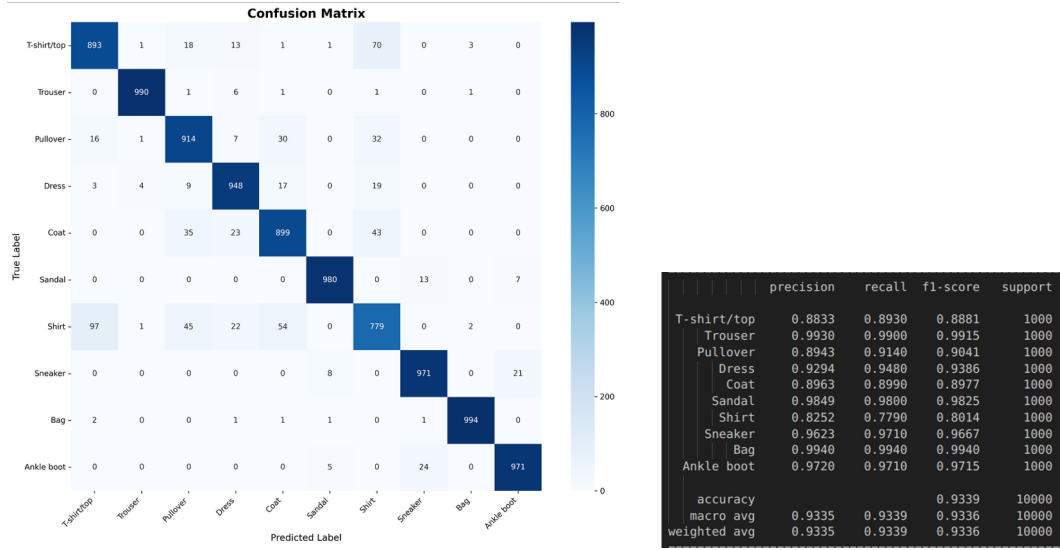
PCA was applied to the flattened image vectors to reduce dimensionality before training classical models. We experimented with varying numbers of components to find the optimal trade-off between information preservation and noise reduction.

Key findings:

- 84 PCA components achieved optimal performance for classical models
- Higher dimensions reintroduced noise, degrading classifier performance
- PCA preprocessing significantly reduced training time for SVM and other classical models

6.2 t-SNE Visualization

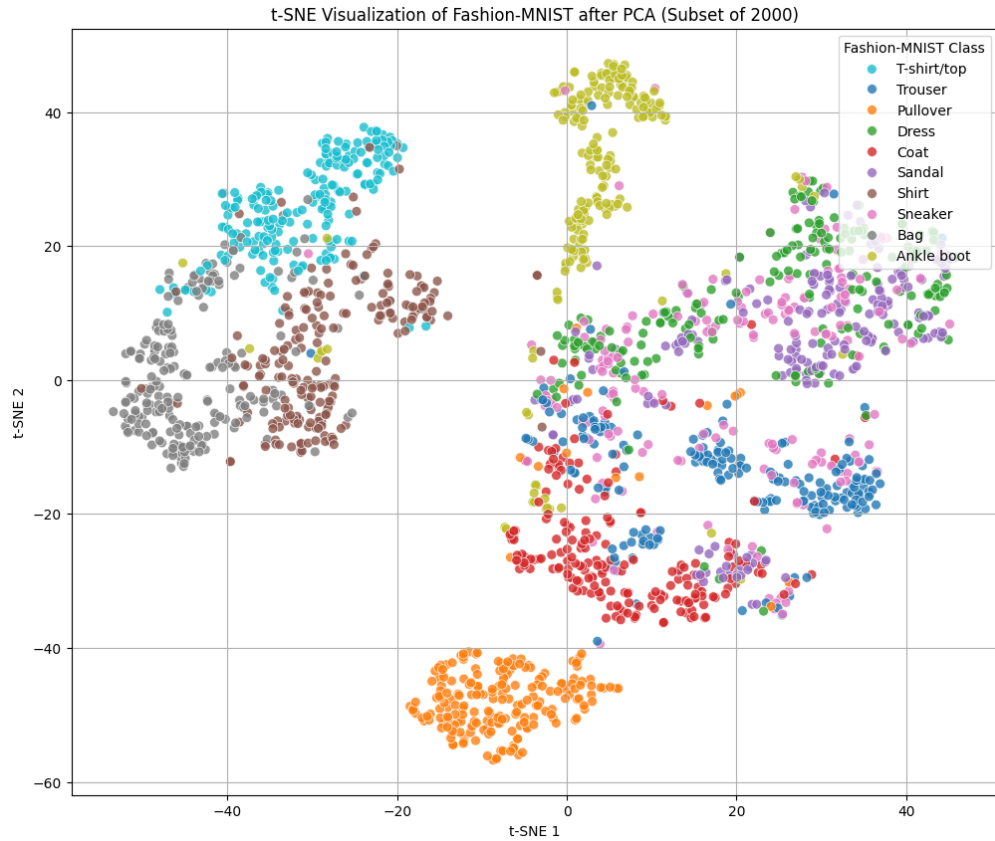
t-SNE groups similar items together, so tight clusters mean the images in that class look visually alike. Well-separated clusters = distinct item types (e.g., shoes vs. bags). Overlapping clusters = visually



(a) Confusion Matrix for CNN Setting 2 (b) Per-Class Metrics Summary

Figure 15: CNN Setting 2 — Confusion Matrix (left) and Per-Class Metrics Table (right).

similar classes (e.g., shirts, pullovers, coats). PCA was used first to reduce noise and speed up t-SNE. Overall, the plot shows how Fashion-MNIST classes naturally cluster based on visual similarity.



t-SNE (t-distributed Stochastic Neighbor Embedding) was used to visualize the high-dimensional feature space in 2D. The visualization revealed:

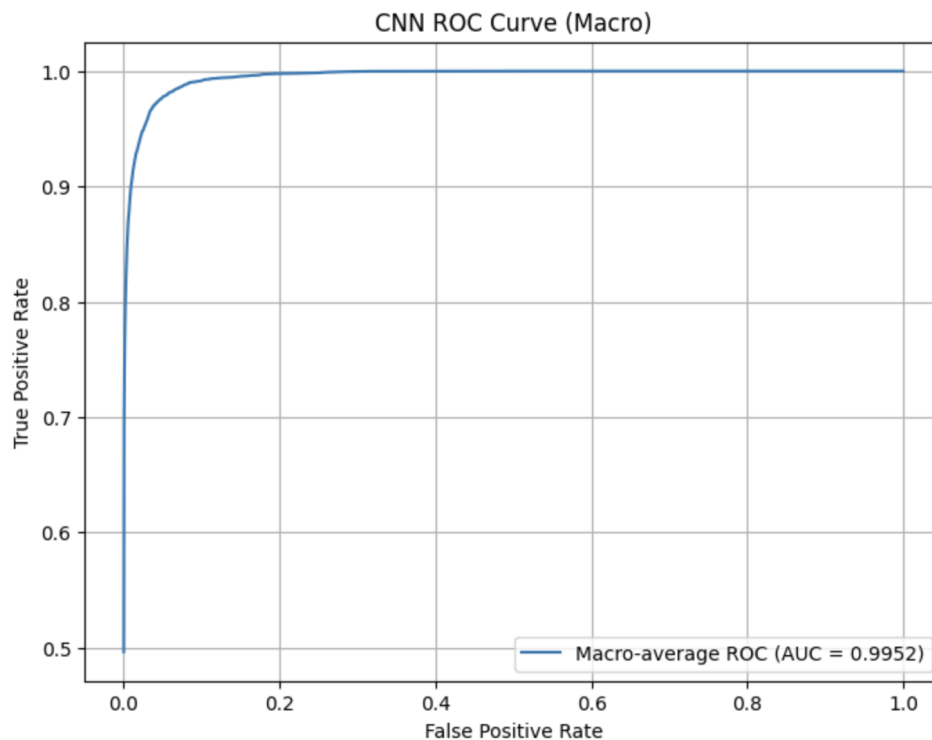
- Clear separation between footwear (Sandal, Sneaker, Ankle boot) and clothing classes
- Significant overlap between Shirt, T-shirt, Pullover, and Coat clusters
- Distinct clusters for Trouser and Bag classes

These observations align with confusion matrix patterns, confirming that class confusion arises from genuine feature-space proximity rather than model limitations.

This plot shows t-SNE applied to Fashion-MNIST features after PCA. Each point is an image; each color is a clothing class.

6.3 ROC Curve

This ROC curve indicates that your Convolutional Neural Network (CNN) model is performing exceptionally well, achieving near-perfect classification results across all classes.



7 Overfitting and Underfitting Analysis

7.1 Classical Models

Random Forest: Demonstrated minimal overfitting due to ensemble averaging and bagging. The gap between training and validation accuracy remained small ($< 3\%$).

SVM: Performance highly dependent on regularization parameter C . Optimal values prevented overfitting while maintaining strong generalization.

7.2 Neural Networks

MLP: Prone to rapid overfitting without regularization. Dropout and early stopping were essential for generalization.

CNN Setting 1: Moderate train-validation gap (91.67% vs 90.23%) indicating slight overfitting but acceptable generalization.

CNN Setting 2: Improved generalization through:

- Dropout (0.3) for regularization
- Weight decay (10^{-4}) in AdamW optimizer
- Cosine annealing learning rate schedule
- Data augmentation for implicit regularization

The train-validation gap (94.90% vs 93.40%) remained small, indicating effective regularization.

8 Binary Classification: Footwear vs. Non-Footwear

We explored a simplified binary classification task to evaluate model performance on a clearly separable subset.

8.1 Class Grouping

- **Footwear Classes** (Label 1): Sandal (5), Sneaker (7), Ankle boot (9)
- **Non-Footwear Classes** (Label 0): T-shirt/top (0), Trouser (1), Pullover (2), Dress (3), Coat (4), Shirt (6), Bag (8)

8.2 Results

CNNs achieved near-perfect binary classification accuracy ($>99\%$), demonstrating that the primary challenge in Fashion-MNIST lies in distinguishing between similar clothing types rather than broad category separation.

9 Team Contributions

- **Aayushi Saxena:** Responsible for project initialization, including the import of core Python libraries, raw data loading, exploration, and fundamental data preprocessing (normalization and reshaping). Implemented MLP with two settings and CNN models; performed dimensionality reduction using PCA and t-SNE; designed and executed the auxiliary binary-grouping task; generated ROC curves and classification reports; and contributed to ppt slide preparation.
- **Akanksha Tyagi:** focused on advanced model development and visualization, including designing and fine-tuning FCNN and CNN architectures (with two variations each) and implementing an SVM with RBF kernel supported by PCA. It also involved generating and interpreting key performance metrics such as confusion matrices and classification reports. Additionally, a Binary Classification experiment (Footwear vs. Non-Footwear) was conducted to assess model behavior on a more clearly separable subset of the Fashion-MNIST dataset.
- **Ved Shukla:** Spearheaded the comprehensive model evaluation and reporting effort. contributions included implementing and benchmarking traditional machine learning models, specifically Random Forest and XGBoost, for classification performance comparison. He also generated and interpreted final performance metrics (Confusion Matrix, Classification Reports, Precision, Recall, F1-Score) for these models. Preparation of visual aids for slides

All team members contributed equally to experimental design, result interpretation, and final presentation preparation.

10 Use of Generative AI

As required by project guidelines, we explicitly disclose the use of Generative AI tools. ChatGPT was used *only* for:

- Formatting assistance and LaTeX debugging
- Grammar and clarity improvements in report writing
- Explaining concepts during learning

All code implementation, experimental design, hyperparameter tuning, model training, and result analysis were performed independently by the team members. No AI-generated code was used in the final implementation.

11 Conclusion

This comprehensive study of Fashion-MNIST classification yielded several key insights:

1. **CNN Superiority:** Convolutional Neural Networks significantly outperform classical machine learning methods, achieving 93.39% accuracy compared to 87.44% for Random Forest. The spatial feature extraction capability of CNNs is essential for image classification tasks.
2. **Regularization Importance:** Advanced regularization techniques (dropout, weight decay, data augmentation) and learning rate scheduling (cosine annealing) substantially improve CNN generalization, reducing the train-test gap.
3. **Class-Specific Challenges:** The Shirt class consistently shows the lowest performance across all models due to high visual similarity with T-shirt/top. This represents an inherent limitation of the dataset rather than a model deficiency.
4. **Dimensionality Reduction:** PCA with 84 components provides optimal feature compression for classical models, improving training efficiency without sacrificing accuracy.
5. **Binary Classification:** The footwear vs. non-footwear task is easily separable, with all models achieving >97% accuracy, confirming that the primary challenge lies in fine-grained clothing classification.
6. **Classical Model Viability:** Despite lower accuracy, Random Forest and XGBoost remain competitive baselines with faster training times and no GPU requirements, making them suitable for resource-constrained environments.

Future work could explore more advanced architectures (ResNet, EfficientNet), attention mechanisms, or self-supervised pre-training to further improve performance on challenging classes.

References

- [1] Xiao, H., Rasul, K., & Vollgraf, R. (2017). *Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms*. arXiv preprint arXiv:1708.07747. :contentReference[oaicite:2]index=2
- [2] Cortes, C., & Vapnik, V. (1995). *Support-Vector Networks*. Machine Learning, 20(3), 273–297.
- [3] Agarap, A. F. (2017). *An Architecture Combining Convolutional Neural Network (CNN) and Support Vector Machine (SVM) for Image Classification*. arXiv preprint arXiv:1712.03541. :contentReference[oaicite:3]index=3
- [4] Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). *Dropout: A Simple Way to Prevent Neural Networks from Overfitting*. Journal of Machine Learning Research, 15(1), 1929–1958.
- [5] Loshchilov, I., & Hutter, F. (2019). *Decoupled Weight Decay Regularization*. International Conference on Learning Representations.
- [6] Breiman, L. (2001). *Random Forests*. Machine Learning, 45(1), 5–32.
- [7] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press. (See Ch. 7 — Regularization and Optimization)